

TIMESTAMP: 2026-04-18T04:00:00Z

ENGINE:

 NODE.JS_EXPRESS_REACT_VITE

 PERSISTENCE:

SQLITE3_KNEX_SINGLEFILE

 AUTH: JWT_STATELESS_BCRYPT

MEDOS_PRD_V3.0::SYSTEM_MANIFEST

TARGET_AUDIENCE: DEVELOPER_NUCLEUS | STATUS: VERIFIED_DEPLOYMENT

1.0 DATABASE_SCHEMA_MAPPINGS

Defines the persistent graph structure across exactly 19 connected entities. UUID v4 acts as primary index allocation logic across all tables.

ENTITY::patients

FIELD	TYPE	CONSTRAINTS / NOTES
id	UUID	PRIMARY_KEY
uhid	VARCHAR	UNIQUE, AUTO-INCREMENT VIA MAX()
name	VARCHAR	NOT_NULL
age	INT	VALIDATE: 0-150
dpdp_consent	BOOLEAN	1/0 INT IN SQLITE
outstanding	DECIMAL	AUTO_SYCNED_REDUX [Balance = Billed-Paid]

ENTITY::encounters

FIELD	TYPE	CONSTRAINTS / NOTES
id	UUID	PRIMARY_KEY
patient_id	FK_UUID	REF: patients.id
doctor_id	FK_UUID	REF: users.id
status	ENUM	[open, signed, cancelled]
vitals_json	TEXT	JSON_PAYLOAD
ai_note	TEXT	MUTABLE_UNTIL_SIGNED

ENTITY::admissions

FIELD	TYPE	CONSTRAINTS / NOTES
id	UUID	PRIMARY_KEY
patient_id	FK_UUID	REF: patients.id

ENTITY::medicine_catalog & stock_transactions

FIELD	TYPE	CONSTRAINTS / NOTES
medicine_id	UUID	REF: catalog.id

room_id	FK_UUID	REF: rooms.id		
status	ENUM	[admitted, discharged, transferred]	transaction_type	ENUM [in, out, adjustment, return]
room_charges	DECIMAL	days_admitted * rooms.daily_rate	quantity	INT NEGATIVE IF type=='out'
			batch_no / expiry	VARCHAR FEFO ALGORITHM SORT KEY

2.0 API_ROUTER_TOPOLOGY

Express.js routing endpoints. Protected strictly by `auth` and `can(...)` RBAC middlewares.

ENDPOINT	PAYLOAD EXPECTATION	MIDDLEWARE / ROLES	EXECUTION_LOGIC
<code>POST</code> <code>/api/auth/login</code>	<code>{ username, password }</code>	<code>RateLimit(15m/5), None</code>	<code>Bcrypt.compare(p, hash) -> Issue 10h JWT payload.</code>
<code>POST</code> <code>/api/encounters/:id/suggest-medicines</code>	<code>{ disease_description }</code>	<code>auth, can('doctor','admin')</code>	Trigger Anthropic API. Match against <code>diseaseMedicineMap</code> . Join matching hits with <code>medicine_catalog</code> . Return structured array.
<code>POST</code> <code>/api/pharmacy/dispense</code>	<code>{ patient_id, medicine_id, qty }</code>	<code>auth, can('pharmacist','admin')</code>	1) Select all unexpired batches. 2) Deduct quantity based on FEFO. 3) Write to <code>stock_transactions</code> . 4) Post line-item charge to <code>charges</code> table.
<code>PUT</code> <code>/api/admissions/:id/discharge</code>	<code>{ discharge_diagnosis }</code>	<code>auth, can('doctor','admin')</code>	Set status to 'discharged'. Calculate <code>`days = today - admission_date`</code> . Multiply by

```
`room.daily_rate`.  
Update  
`room_charges`.  
Free the room in  
frontend map.
```

3.0 UI_RENDER_TREE (REACT.JS / VITE)

Component hierarchy relying on precise route isolation and Context API based toast notifications.

```
<App>  
  <BrowserRouter>  
    |— <Route path="/login" component={Login} /> // Stateless UI, triggers AuthStore state mutation  
    |— <Guard>  
    |   |— <Layout> // Providers: ToastContext. Contains Sidebar, Header  
    |   |   |— <Sidebar> // Navigation filtered map against user?.role  
    |   |   |   |— <ClockWidget /> // Isolated re-render loop via React.memo interval  
    |   |   |   |— <Header> // Route metadata injector  
    |   |   |   |— <Outlet>  
    |   |   |       |— <Dashboard> // Uses stats payload to render metrics and KPI trends.  
    |   |   |       |— <OPD> // Encounter list navigation + Detail tabs (Vitals, Prescriptions, A  
    |   |   |       |— <IPD> // Room matrix loop mapped against current admissions.  
    |   |   |       |— <Pharmacy> // Data grid with expandable rows to `transaction_history`.  
    |   |   |       |— <Billing> // Multi-ledger aggregation (Charges -> Invoices -> Payments).  
    |   |   |       |— <Reports> // Read-only analytic charts using Recharts mapped array.
```

4.0 GLOBAL_STATE_MANAGEMENT

Zustand acts as the client-side memory volatile store representing the JWT session.

```
const useAuthStore = create((set) => ({  
  user: null, // Hydrated from decoded JWT payload  
  token: null, // Active Bearer token  
  login: (token, user) => { localStorage.setItem('token', token); set({ token, user }); },  
  logout: () => { localStorage.removeItem('token'); set({ token: null, user: null }); },  
}))
```

5.0 CRITICAL_LOGIC_FUNCTIONS

5.1 PATIENT_LEDGER_SYNC

Financial integrity requires auto-synchronization on any insert/update mutation associated with billing objects.

`function syncPatientBalance(patient_id)` executes:

- `total_billed` = SUM(invoices.total_amount) where patient_id = X
- `total_paid` = SUM(payments.amount) where patient_id = X and status = 'success'
- `outstanding` = total_billed - total_paid
- Updates `patients` record instantly.

5.2 FEFO_INVENTORY_ALGORITHM

Medicine dispensing utilizes a First-Expired-First-Out cascading subtraction matrix.

```
const batches = DB.select('id', 'remaining_qty', 'expiry_date').orderBy('expiry_date', 'ASC')
let required = payload.quantity;
for (let b of batches) {
  if (required <= 0) break;
  let ded = Math.min(required, b.remaining_qty);
  b.remaining_qty -= ded;
  required -= ded;
  SQL.update({ remaining_qty: b.remaining_qty });
}
if (required > 0) throw "INSUFFICIENT_STOCK_EXCEPTION";
```

6.0 CSS_DESIGN_SYSTEM (UI_TOKENS)

The UI is completely constrained by root variables in `index.css`. No hardcoded hex values are permitted in React inline styles.

```
:root {
  --bg-deep: #06060b;
  --surface: rgba(20, 20, 34, 0.7);
  --surface-2: rgba(30, 30, 50, 0.8);
  --border: rgba(255, 255, 255, 0.08);
  --text-white: #ffffff;
  --text: #e2e8f0;
  --primary: #6366f1;
  --success: #10b981;
}
/* Glassmorphism definition */
.card, .modal {
  backdrop-filter: blur(24px);
  -webkit-backdrop-filter: blur(24px);
  box-shadow: 0 8px 32px rgba(0,0,0,0.4);
```

Interaction Constants: Buttons utilize `transform: translateY(-1px)` and `filter: brightness(1.1)` on hover to simulate tactile floating feedback. Empty states necessitate a centralized layout flexbox paradigm.

Z-Index Heirarchy:

- `--z-sidebar`: 40
- `--z-header`: 30
- `--z-modal-overlay`: 100
- `--z-toast`: 200

```
}
```

7.0 DEPLOYMENT_TOPOLOGY & SECURITY_PARAMETERS

The system is verified to run entirely stateless on the Node.js boundary, leveraging a localized SQLite file `backend/medos.db` acting as the primary single source of truth. Security mechanisms integrated include:

- Strict parameterized execution via Knex to nullify all vector SQL injections.
- Input boundary assertions executing on POST/PUT requests prior to query construction.
- Express Rate Limiter applied against identity interfaces to mitigate brute forcing.